<table>
<tr><th>Title</th><th>Data Manipulation and Analysis using the Starwars Dataset in R</th></tr>
<tr><td>Problem Statement</td><td>Basic Data Manipulation with Pandas: Create a DataFrame with columns Name, Age, and City containing data for five individuals. Perform the following operations: select only the Name and Age columns, filter rows where Age is greater than 25, add a new column Country with a default value, and sort the DataFrame by Age in descending order. Finally, calculate the average age of the individuals.</td></tr>
<tr><td>Objectives</td><td>To understand how to create a Pandas DataFrame from a Python
dictionary.
To learn column selection techniques using Pandas.
To apply filtering conditions on DataFrame rows.
To create and populate new columns in a DataFrame.
To sort data based on column values.
To perform basic statistical analysis using Pandas aggregate functions.</td></tr>
<tr><td>Dataset URL Link / File</td><td><table><tr><td>Name</td><td>Age</td><td>City</td></tr><tr><td>Alice</td><td>23</td><td>New York</td></tr><tr><td>Bob</td><td>30</td><td>London</td></tr><tr><td>Charlie</td><td>28</td><td>Paris</td></tr><tr><td>David</td><td>35</td><td>Tokyo</td></tr><tr><td>Emma</td><td>22</td><td>Sydney</td></tr></table></td></tr>
<tr><td>Functions Details</td><td><table><tr><th>Sl. No.</th><th>Function</th><th>Package</th><th>Description of function</th></tr><tr><td>1</td><td>DataFrame()</td><td>pandas</td><td>Creates a DataFrame</td></tr><tr><td>2</td><td>sort_values()</td><td>pandas</td><td>Sorts DataFrame rows</td></tr><tr><td>3</td><td>mean()</td><td>pandas</td><td>Calculates the average value</td></tr><tr><td>4</td><td>sort_values()</td><td>pandas</td><td>Sorts DataFrame rows</td></tr></table></td></tr>
<tr><td>Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model</td><td>Step 1
Import the pandas library and create a dictionary containing Name, Age and City Data

Step 2
Convert the dictionary into a pandas DataFrame

Step 3
Select only the Name and Age columns from the DataFrame.

Step 4
Filter the DataFrame to display individuals whose age is greater than 25.</td></tr>
</table>

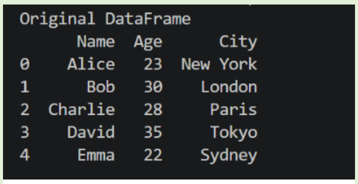
	Step 5 Add a new column named Country and assign a default value of India. **Step 6** Sort the DataFrame in descending order based on the Age column. **Step 7** Calculate the average age of all individuals using the mean() function. **Step 8** – Display all intermediate and final results.

Program

```
# Load Necessary Packages
import pandas as pd

# Step 1: Create Dataset
dataframe1 = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Emma"],
    "Age": [23, 30, 28, 35, 22],
    "City": ["New York", "London", "Paris", "Tokyo", "Sydney"]
}

# Step 2: Create DataFrame
df = pd.DataFrame(dataframe1)

print("Original DataFrame")
print(df)
print()

# Step 3: Select Name and Age Columns
name_age = df[["Name", "Age"]]

print("Name and Age")
print(name_age)
print()

# Step 4: Filter Rows where Age > 25
filtered_df = df[df["Age"] > 25]

print("Age > 25")
print(filtered_df)
print()
```

```
# Step 5: Add Country Column
df["Country"] = "India"

print("Country Added")
print(df)
print()

# Step 6: Sort by Age in Descending Order
sorted_df = df.sort_values(by="Age", ascending=False)

print("Sorted Data")
print(sorted_df)
print()

# Step 7: Calculate Mean Age
mean_df = df["Age"].mean()

print("Mean Age")
print(mean_df)
```

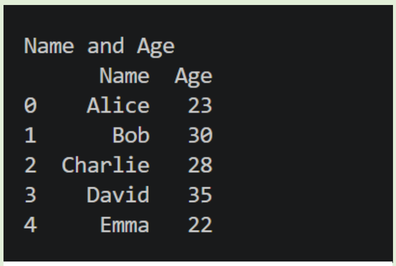
Output Snapshots With Explanation/ Observations



```
Original DataFrame
      Name  Age      City
0    Alice   23  New York
1      Bob   30    London
2  Charlie   28     Paris
3    David   35     Tokyo
4     Emma   22    Sydney
```

Figure 1.1 Original Dataframe

Figure 1.1 displays the original DataFrame created from a Python dictionary containing information about five individuals. The DataFrame consists of three columns: Name, Age, and City, where each row represents a different individual and their corresponding details. The pd.DataFrame() function converts the dictionary data into a structured tabular format, making it easier to store, view, and manipulate data using Pandas. This DataFrame serves as the starting dataset for all subsequent operations such as column selection, filtering, adding new columns, sorting, and calculating statistical measures.



```
Name and Age
      Name  Age
0    Alice   23
1      Bob   30
2  Charlie   28
3    David   35
4     Emma   22
```

Figure 1.2 Selected Name and Age Columns

Figure 1.2 shows the DataFrame after selecting only the Name and Age columns from the original dataset using Pandas column indexing. This operation extracts the required columns and creates a new DataFrame named name_age, removing the City column from the view. Such column selection helps focus on relevant attributes for analysis and reduces unnecessary data processing.

	Name	Age	City
1	Bob	30	London
2	Charlie	28	Paris
3	David	35	Tokyo

Figure 1.3: Filtered Data (Age > 25)

Figure 1.3 presents the filtered DataFrame containing only those individuals whose Age is greater than 25. This filtering operation is performed using a conditional expression in Pandas, where the Age column is checked against the condition Age > 25. As a result, only the rows satisfying the condition are retained in the new DataFrame filtered_df. Figure 1.3 operation demonstrates how Pandas can be used to extract specific records based on user-defined criteria, which is a common requirement in data analysis and data preprocessing tasks.

	Name	Age	City	Country
0	Alice	23	New York	India
1	Bob	30	London	India
2	Charlie	28	Paris	India
3	David	35	Tokyo	India
4	Emma	22	Sydney	India

Figure 1.4: DataFrame with Country Column

Figure 1.4 displays the DataFrame after adding a new column named Country with a default value of "India" for all records. This operation is performed using Pandas column assignment, where the same value is assigned to every row in the newly created column. Adding new columns is a common data transformation technique used to enrich datasets with additional information. Figure 1.4 output confirms that the Country column has been successfully added to the DataFrame, and each individual has been assigned the default value India. The enriched dataset is now ready for further operations such as sorting and statistical analysis.

	Name	Age	City	Country
3	David	35	Tokyo	India
1	Bob	30	London	India
2	Charlie	28	Paris	India
0	Alice	23	New York	India
4	Emma	22	Sydney	India

Figure 1.5 Sorted Data by Age (Descending Order)

Figure 1.5 illustrates the DataFrame after sorting the records based on the Age column in descending order using the Pandas sort_values() function. In this operation, individuals are arranged from the highest age to the lowest age. Sorting helps organize data systematically and makes it easier to identify records with the largest or smallest values. Figure 1.5 output confirms that the DataFrame has been successfully sorted in descending order of Age, placing the oldest individual at the top and the youngest individual at the bottom. Such sorting operations are commonly used in data analysis to rank, compare, and organize records efficiently.

```
Mean of the average age  27.6
```

Figure 1.6 Average Age of Individuals

Figure 1.6 presents the average age of all individuals in the dataset, calculated using the Pandas mean() function. The average (mean) age is obtained by summing all the age values and dividing the total by the number of individuals in the DataFrame. This operation demonstrates the use of aggregate statistical functions in Pandas for summarizing numerical data. Figure 1.6 output confirms that the Pandas mean() function has successfully computed the average age of the five individuals as 27.6 years. Such aggregate functions are widely used in data analysis to summarize and interpret numerical information efficiently.

Conclusions	This experiment successfully demonstrated the use of **Pandas for basic data manipulation and analysis**. A DataFrame containing the details of five individuals was created using the columns **Name, Age, and City**. Various DataFrame operations were then performed, including selecting specific columns, filtering rows based on a condition, adding a new column with default values, sorting records, and calculating statistical measures. These operations showcased the flexibility and efficiency of Pandas in handling tabular data. The results showed that: • The **Name** and **Age** columns could be extracted independently for focused analysis. • Filtering enabled the identification of individuals whose **Age was greater than 25 years**. • A new **Country** column was successfully added to enrich the dataset. • Sorting arranged the records in descending order of age, making it easy to identify the oldest and youngest individuals. • The **average age** of the individuals was calculated as **27.6 years**, demonstrating the use of aggregate functions in data analysis. Overall, this exercise provided a strong foundation in **DataFrame creation, data selection, filtering, transformation, sorting, and statistical analysis using Pandas**. These fundamental data manipulation techniques are widely used in **Data Science, Machine Learning, Data Analytics, and Business Intelligence** applications, making them essential skills for working with real-world datasets.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018 2. Pandas Documentation: https://pandas.pydata.org/docs/ 3. Python Documentation: https://docs.python.org/3/ 4. Python Program (Google Colab link /Github link) https://colab.research.google.com/drive/1sjdIVacPTNg2JO_nGS8gic99hGEtMuRe?usp=sharing 5. Class lecture link https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view